

ELARA.AI

Web Application Architecture and Technical Blueprint

Production architecture for the browser application, API boundary, asynchronous verification worker, research services, data systems, deployment, and report experience

Core principle

Elara.ai separates the browser interface, protected API, long-running verification worker, and durable data systems. This keeps private credentials on the server, lets research jobs fail or scale independently, and gives users live progress without blocking web requests.

Consolidated proposal - Version 1.0

June 2026

Contents

1	Product purpose and architectural boundaries
2	High-level system architecture and communication rules
3	Verification request lifecycle and run states
4	Frontend application
5	Backend API and application boundary
6	Multi-agent verification worker and deterministic services
7	Research and extraction subsystem
8	Storage, database model, and data lifecycle
9	Background processing and live progress
10	Authentication, authorization, privacy, and security
11	Deployment, monitoring, and CI/CD
12	Repository structure and ownership
13	Routes and application surfaces
14	Report output, score presentation, and API contract
15	Complete technology stack and implementation order

Document scope

This blueprint consolidates the complete Elara.ai architecture and technical stack. It defines subsystem ownership, interfaces, asynchronous execution, agent workflow, storage, security, deployment, repository organization, user-facing routes, and the recommended implementation sequence.

1. Product purpose and architectural boundaries

Elara.ai is a web-based fact-checking and evidence-analysis platform. Users submit a claim, quotation, news article, URL, pasted text, or source document. The system decomposes the input, performs supporting and contradicting research, analyzes source provenance, audits numbers and quotations, and returns a citation-grounded report.

Primary product responsibilities

- Provide a responsive browser interface for submissions, live research progress, and completed reports.
- Run current-information research and evidence extraction without exposing provider credentials to the browser.
- Separate long-running verification jobs from short API requests.
- Store claims, sources, passages, evidence, scores, events, versions, feedback, and exports in durable systems.
- Show exactly how the result was reached without exposing private chain-of-thought or internal reasoning transcripts.
- Make reports shareable, reproducible, versioned, and correctable.

System boundaries

Layer	Owns	Does not own
Next.js web app	Pages, routing, interaction, session-aware UI, responsive report views	Provider keys, agent prompts, direct database or storage credentials
FastAPI backend	Authentication checks, validation, run lifecycle, upload and report APIs, access control	Long-running research inside request handlers
Verification worker	Agents, extraction, provenance, deterministic scoring, numerical audits, citation audits	Rendering the browser interface
PostgreSQL + object storage	Durable application records, source data, permitted snapshots, uploads, and exports	Transient queue orchestration
Redis	Queues, cache, locks, rate limits, and live progress	The long-term source of record

Why this separation matters

A browser bundle can be inspected by any user. Private credentials and privileged logic remain behind FastAPI. The worker can retry, fail, scale, time out, or be cancelled independently without freezing the web request.

2. High-level system architecture and communication rules

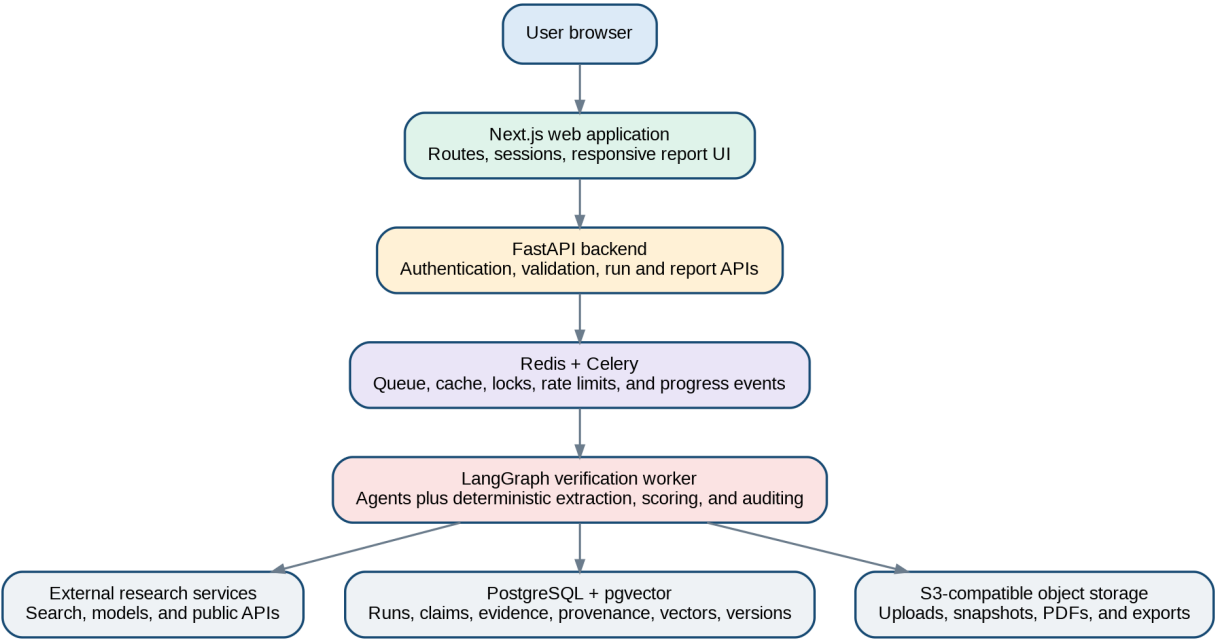


Figure 1. High-level Elara.ai web application architecture.

Responsibilities by layer

1. The user interacts with the Next.js application through HTTPS.
2. Next.js renders routes and interactive components and calls FastAPI for protected data and actions.
3. FastAPI authenticates and validates the request, creates a verification run, and enqueues a background job.
4. Redis transports the job to a Celery worker and carries transient locks, cache entries, rate limits, and progress events.
5. The LangGraph worker coordinates language-model agents and deterministic research, extraction, provenance, scoring, and auditing services.
6. PostgreSQL stores relational data and vectors through pgvector; object storage stores larger uploads, permitted snapshots, and exports.
7. The browser receives progress through Server-Sent Events and opens the completed report after the durable run passes citation auditing.

Communication rules

Connection	Protocol	Purpose
Browser -> Next.js / FastAPI	HTTPS	Pages, authenticated API calls, uploads, actions, and report retrieval
Browser <- FastAPI	Server-Sent Events	One-way live progress stream
FastAPI -> Redis	Queue command	Create asynchronous verification jobs
Worker <-> Redis	Queue and events	Claim jobs, retries, locks, status, cache, rate limits, and progress
Worker <-> PostgreSQL	SQL	Runs, claims, sources, passages, evidence, scores, versions, and vectors
Worker <-> object storage	S3-compatible API	Uploads, source snapshots, PDFs, and exported reports
Worker <-> external providers	HTTPS / provider APIs	Search, models, public APIs, and source retrieval

3. Verification request lifecycle and run states



Figure 2. Request lifecycle from submission to completed report.

Lifecycle details

- 1. The user submits text, an article URL, quotation, or supported file through the web interface.
- 2. FastAPI authenticates the user, validates the request schema, checks account and research-depth limits, and normalizes the input type.
- 3. The API creates a verification run with status QUEUED and returns the run identifier immediately.
- 4. A Celery worker receives the job and invokes the LangGraph workflow.
- 5. Each workflow node stores public progress events in Redis and durable outputs in PostgreSQL.
- 6. The Next.js page subscribes to an SSE endpoint and updates the interface as observable work completes.
- 7. When the report passes citation auditing, the durable run changes to COMPLETED and the report route becomes available.

Suggested run states

QUEUED
VALIDATING
DECOMPOSING
RESEARCHING
EXTRACTING
ANALYZING_PROVENANCE
SCORING
SYNTHESIZING
AUDITING
COMPLETED
FAILED
CANCELLED

User experience rule

Display observable actions such as sources found, passages extracted, derivative sources grouped, calculations completed, and citations audited. Never expose hidden chain-of-thought or private reasoning transcripts.

4. Frontend application

Selected frontend stack

Category	Technology	Role
Framework	Next.js App Router	Routing, layouts, server rendering, metadata, and web deployment
Language	TypeScript	Type-safe UI and API contracts
UI library	React	Interactive components
Styling	Tailwind CSS	Responsive layout and design tokens
Components	shadcn/ui	Accessible dialogs, tabs, menus, cards, drawers, and forms
Server data	TanStack Query	Fetching, caching, retries, invalidation, and loading states
Local UI state	Zustand	Transient tabs, filters, selected evidence, panels, and workspace state
Forms	React Hook Form	Efficient form handling
Validation	Zod	Client-side schemas and shared contracts
Source graph	React Flow	Interactive provenance and dependency graph
Charts	Recharts	Score, coverage, evidence, and numerical visualizations
Icons	Lucide React	Consistent interface iconography

Frontend responsibilities

- Render the public landing page, methodology, account pages, live verification view, history, settings, and report workspace.
- Validate inputs before submission while treating FastAPI as the final authority.
- Use multi-panel layouts on desktop and drawers or tabs on smaller screens.
- Use TanStack Query for server-owned data rather than duplicating reports in Zustand.
- Use Zustand only for transient interface choices.
- Stream progress through SSE and reconnect safely after temporary network interruption.
- Keep only non-sensitive interface preferences in localStorage.

Recommended interface regions

Area	Contents
Submission workspace	Claim input, article URL, quotation, upload control, research-depth selection, and validation messages
Live research view	Workflow status, source counts, completed steps, inaccessible-source notices, and cancellation controls
Report workspace	Verdict, scores, atomic claims, evidence, source graph, numerical audit, context, and methodology
Source drawer	Exact passage, metadata, page/section, dependency cluster, snapshot information, and external source link
History and saved reports	Search, filters, saved reports, rerun, export, sharing, and deletion controls

5. Backend API and application boundary

Selected backend stack

Category	Technology	Role
API framework	FastAPI	REST endpoints, streaming, authentication boundary, and file uploads
Language	Python	Backend services and integration with the worker
Validation	Pydantic	Typed requests, responses, and agent schemas
ORM	SQLAlchemy	Relational data access
Migrations	Alembic	Versioned database changes
Server	Uvicorn	ASGI runtime
HTTP requests	httpx	Asynchronous provider and public-source requests
Live progress	Server-Sent Events	Progress delivery to browsers
File uploads	FastAPI upload endpoints	Secure ingestion of supported documents

Backend service boundaries

- Authenticate every protected request and determine the current user.
- Validate input sizes, file types, URLs, research depth, and account limits.
- Create, cancel, retry, save, export, share, and delete verification runs.
- Issue short-lived access to permitted source snapshots rather than exposing storage credentials.
- Read durable report data from PostgreSQL and transient progress from Redis.
- Return stable, versioned response schemas for the frontend and future clients.

Core API groups

```
POST    /v1/verifications
GET     /v1/verifications/{run_id}
GET     /v1/verifications/{run_id}/events
POST    /v1/verifications/{run_id}/cancel
POST    /v1/verifications/{run_id}/retry
GET     /v1/verifications/{run_id}/report
GET     /v1/verifications/{run_id}/sources
GET     /v1/verifications/{run_id}/source-graph
GET     /v1/history
POST    /v1/verifications/{run_id}/feedback
```

6. Multi-agent verification worker and deterministic services

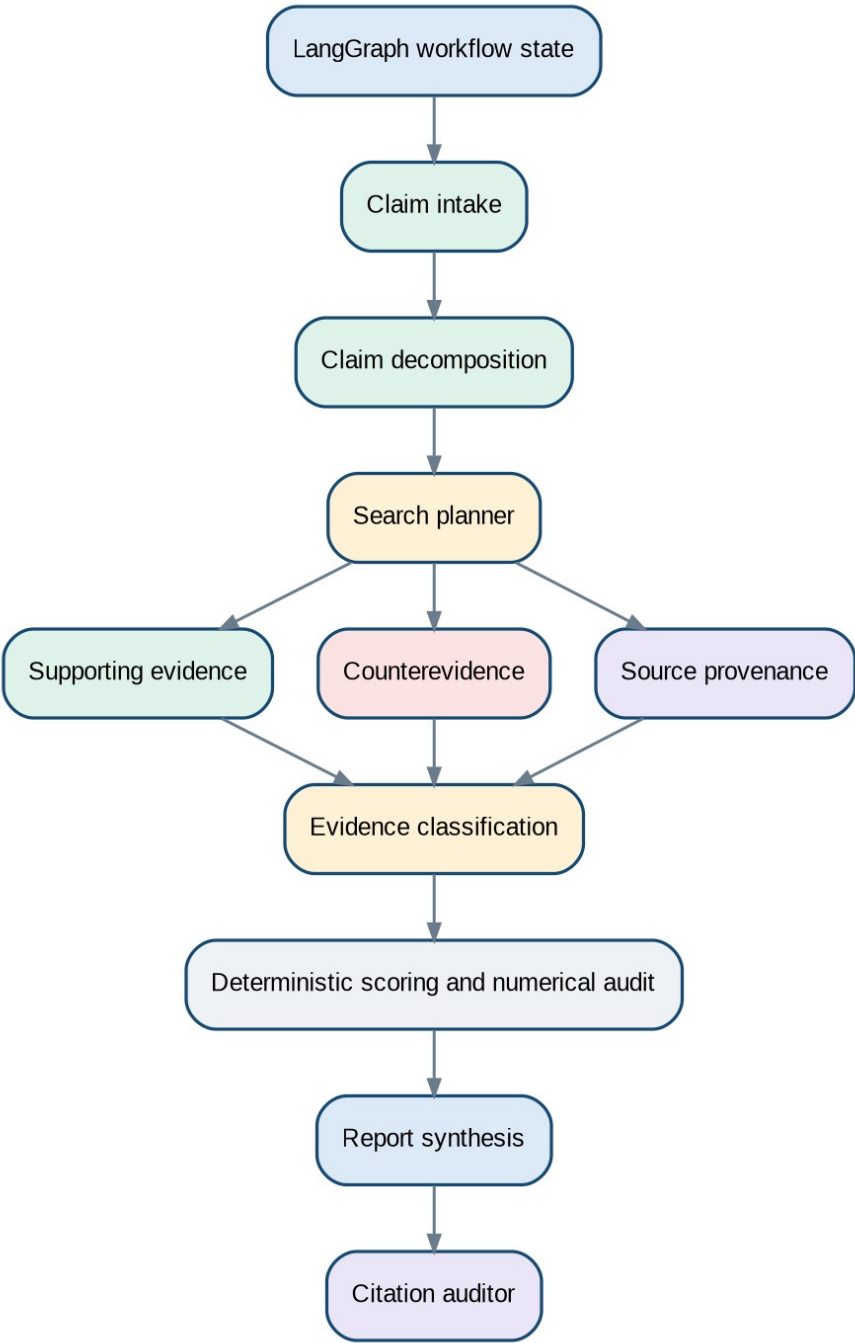


Figure 3. Controlled agent workflow with deterministic scoring and citation auditing.

Selected agent stack

Category	Technology	Role
Workflow orchestration	LangGraph	Stateful, conditional verification workflow
LLM integration	OpenAI Responses API	Extraction, classification, planning, and evidence-grounded synthesis
Current research	Web-search tool or API	Discover up-to-date public sources
Structured outputs	Pydantic schemas	Predictable contracts between workflow nodes

Category	Technology	Role
Embeddings	OpenAI embeddings	Similarity, passage retrieval, and duplicate clustering
Agent tracing	LangSmith	Run inspection, debugging, and trace analysis
Evaluation	Custom Python evaluators + LangSmith	Benchmarks, graders, regression tests, and quality measurement

Agent responsibilities

Agent	Primary responsibility
Claim Intake	Classify input, identify speakers, entities, dates, locations, metrics, and fact-checkability.
Claim Decomposition	Split complex statements into independently testable atomic claims.
Search Planner	Generate primary, support, contradiction, attribution, definition, correction, and context queries.
Supporting Evidence	Find the strongest evidence consistent with the claim.
Counterevidence	Find contradictions, corrections, omitted context, alternative definitions, and competing explanations.
Source Provenance	Trace information origins and group dependent reporting.
Evidence Classification	Assign stance, relevance, directness, authority, temporal fit, and extraction certainty.
Report Synthesis	Explain approved findings without inventing evidence.
Citation Auditor	Confirm that every factual report sentence is supported by the cited passage.

Deterministic control

Calculations, hashes, URL validation, exact thresholds, dependency multipliers, rate limits, unit conversions, and citation-presence checks are normal code. Agents are used where language understanding is required, not where exact computation is available.

7. Research and extraction subsystem

Selected retrieval stack

Category	Technology	Purpose
Discovery	Search API / model web search	Candidate URL discovery and query diversification
Web requests	httpx	Fast asynchronous static requests with timeouts and limits
Main article extraction	Trafilatura	Readable article body and metadata
HTML parsing	Beautiful Soup	Links, structured elements, corrections, metadata, and custom rules
JavaScript fallback	Playwright	Client-rendered pages only when static retrieval is insufficient
PDF processing	PyMuPDF	Page-aware text, links, metadata, and layout extraction
Data tables	pandas	Structured-table normalization and analysis
Exact arithmetic	Python Decimal	Reproducible percentages, ratios, totals, and comparisons
Date parsing	dateparser	Publication dates and relative date expressions
Vector retrieval	OpenAI embeddings + pgvector	Semantic matching and similarity clustering
Duplicate detection	Content hashes + embeddings	Exact and near-duplicate source detection

Extraction pipeline and fallback order

1. Validate the target URL and block internal or unsafe destinations.
2. Fetch with timeouts, redirect and response-size limits, rate controls, and a descriptive user agent.
3. Extract title, author, publication/update dates, article text, links, quotations, tables, and corrections.
4. Create a content hash and compare against existing sources.
5. Split accepted content into passages and generate embeddings.
6. Store the exact passage, page/section, metadata, and snapshot information used by the final report.

Fallback order

1. httpx + Trafilatura
2. Beautiful Soup custom extraction
3. Playwright rendered page
4. PyMuPDF when the response is a PDF
5. Mark inaccessible and continue with alternative sources

Architecture boundary

The complete methodology for query planning, secure retrieval, evidence scoring, quote verification, and verdict logic is defined in the companion Elara.ai Verification and Targeted Retrieval Methodology. This section defines the production components that execute it.

8. Storage, database model, and data lifecycle

Storage categories

Category	Technology	Stored data
Main database	PostgreSQL	Users, runs, claims, sources, passages, evidence, scores, feedback, events, and versions
Vector search	pgvector	Passage embeddings, similarity search, and duplicate clustering
Cache	Redis	Frequently reused results and transient application data
Job state	Redis	Queued jobs, retries, locks, limits, and progress events
Source snapshots	S3-compatible object storage	Fetches documents, permitted snapshots, and uploads
Report exports	Object storage	Generated PDF, JSON, and future report formats
Browser preferences	localStorage where appropriate	Non-sensitive interface preferences only

Durable data model

```
users
verification_runs
atomic_claims
sources
source_snapshots
source_passages
evidence_items
source_dependencies
calculations
agent_events
user_feedback
methodology_versions
```

Key record fields

```
Source
- id, canonical_url, domain, title, author, publisher
- published_at, updated_at, retrieved_at
- source_type, content_type, access_status
- content_hash, parser_name, parser_version
- snapshot_path, extraction_quality
- correction_status, robots_or_policy_status

Passage
- id, source_id, text, heading_path, page_or_position
- embedding, extraction_certainty

DependencyEdge
- parent_source_id, child_source_id
- relationship, confidence, detection_method
```

Data lifecycle rules

- PostgreSQL is the source of truth for in-progress and completed verification records.
- Redis entries may expire because durable state can be reconstructed from PostgreSQL.
- Object-storage paths are referenced from the database and are never exposed as permanent public credentials.
- Embeddings remain attached to passage identifiers so vector results can be traced to exact text.
- Every report stores model, prompt, workflow, parser, retrieval, and scoring-methodology versions.
- Changed sources create new snapshots rather than overwriting evidence used by earlier reports.
- Deletion workflows remove or anonymize database rows and associated object-storage files according to policy.

9. Background processing and live progress

Category	Technology	Role
Task queue	Celery	Dispatch and retry long-running verification jobs
Queue broker	Redis	Transport jobs to available workers
Worker language	Python	Run agent, extraction, provenance, scoring, and auditing code
Progress events	Redis Streams or pub/sub	Publish user-visible workflow status
Browser delivery	Server-Sent Events	Stream progress to the active run page

Why a worker is required

- A full verification may take much longer than a normal HTTP request.
- Workers can retry individual failures without repeating the entire submission.
- Research capacity can scale independently from web traffic.
- FastAPI remains responsive during expensive extraction or model calls.
- Jobs can be cancelled, timed out, prioritized, resumed, or retried according to policy.

Progress event example

```
event: progress
data: {
  "run_id": "run_123",
  "stage": "EXTRACTING",
  "message": "Extracted 9 of 12 candidate sources",
  "completed_steps": 4,
  "total_steps": 9
}
```

Reliability rule

Progress events are informative, not authoritative. Final status and report content must always be recoverable from PostgreSQL after a browser refresh, connection drop, or Redis restart.

10. Authentication, authorization, privacy, and security

MVP authentication requirements

- Email and password or OAuth through a managed authentication provider.
- Secure HTTP-only session cookies or short-lived access tokens.
- Per-user history and saved-report access controls.
- Usage limits based on account plan and research depth.
- Independent token validation by FastAPI before protected data is returned.

Security controls

Area	Required control
Secrets	Store provider, storage, and database credentials only in server-side secret managers.
Uploads	Limit type and size; scan or reject unsupported content; never execute uploaded files.
URL fetching	Block localhost, private IPs, cloud metadata, unsafe schemes, non-approved ports, and redirect abuse.
Prompt injection	Treat retrieved pages as untrusted evidence and ignore instructions contained inside them.
Authorization	Verify ownership and sharing policy for every run, source, snapshot, export, and saved report.
Rate limiting	Enforce per-user and per-IP limits before expensive work is queued.
Logging	Avoid access tokens, passwords, full private uploads, and unnecessary personal content.
Privacy	Publish retention, deletion, sharing, snapshot, and public-report policies.
Transport and storage	Use TLS in transit and encryption at rest for sensitive records.

Browser security boundary

The Next.js client never calls OpenAI, search providers, PostgreSQL, Redis, or object storage using private credentials. Every privileged action passes through FastAPI or the worker.

11. Deployment, monitoring, and CI/CD

Recommended deployment

Component	Deployment
Next.js frontend	Vercel
FastAPI backend	Container hosting
Celery worker	Separate container service
PostgreSQL	Managed PostgreSQL with pgvector
Redis	Managed Redis
Object storage	S3-compatible service
Monitoring	Sentry
Agent tracing	LangSmith
CI/CD	GitHub Actions

Deployment topology

- Deploy the frontend independently so static and server-rendered assets scale with browser traffic.
- Deploy FastAPI as a stateless service with access to PostgreSQL, Redis, and object storage.
- Deploy one or more workers using the same Python package versions as the API.
- Run database migrations as a controlled deployment step before incompatible code is activated.
- Use separate development, staging, and production environments with isolated credentials and data.

Monitoring signals

Category	Examples
Application health	Error rate, API latency, failed uploads, authentication failures
Worker health	Queue length, job duration, retries, timeouts, failed workflow nodes
Research quality	Extraction failures, inaccessible sources, low evidence yield, citation-audit failures
Cost	Model tokens, search calls, browser fallbacks, storage growth, cost by research depth
Database	Slow queries, connection usage, vector-search latency, migration status
User experience	Time to first progress event, completion latency, report-open errors

GitHub Actions pipeline

1. Install dependencies.
2. Run TypeScript checks and frontend tests.
3. Run Python linting, type checks, and tests.
4. Build Next.js and container images.
5. Run database migration checks.
6. Deploy staging.
7. Execute smoke, security, and evaluation tests.
8. Approve and deploy production.

12. Repository structure and ownership

A monorepo keeps the browser application, API, worker, shared contracts, evaluation suite, and infrastructure changes synchronized.

```
Elara.ai/
|-- apps/
|   |-- web/
|       |-- app/
|           |-- page.tsx
|           |-- verify/
|           |-- reports/
|           |-- history/
|           |-- sources/
|           |-- settings/
|           |-- api/
|       |-- components/
|           |-- claim-input/
|           |-- live-research/
|           |-- verdict/
|           |-- evidence/
|           |-- source-graph/
|           |-- numerical-audit/
|       |-- hooks/
|       |-- stores/
|       |-- lib/
|   |-- api/
|       |-- app/
|           |-- routes/
|           |-- auth/
|           |-- database/
|           |-- models/
|           |-- schemas/
|           |-- services/
|       |-- tests/
|   |-- worker/
|       |-- agents/
|       |-- graph/
|       |-- research/
|       |-- extraction/
|       |-- provenance/
|       |-- scoring/
|       |-- auditing/
|       |-- tasks/
|-- packages/
|   |-- api-client/
|   |-- shared-types/
|   |-- design-tokens/
|-- evals/
|-- infrastructure/
|-- docker-compose.yml
|-- README.md
```

Folder ownership

Folder	Responsibility
apps/web	Browser routes, components, responsive layouts, and frontend data access
apps/api	FastAPI routes, authentication, validation, persistence, reports, and protected snapshot access
apps/worker	LangGraph workflow, agents, retrieval, extraction, provenance, scoring, and audits
packages/api-client	Generated or hand-written typed client used by frontend applications
packages/shared-types	Shared enums, run states, report contracts, and schemas
packages/design-tokens	Colors, spacing, typography, and reusable interface tokens
evals	Benchmarks, graders, baselines, security fixtures, and regression reports
infrastructure	Containers, deployment definitions, environment configuration, and CI/CD

13. Routes and application surfaces

Route	Purpose
/	Public landing page
/verify	Submit a claim, article, quotation, URL, or source document
/verify/{runId}	Live research and progress view
/report/{runId}	Completed verification report
/history	Previous investigations
/saved	Saved reports
/methodology	Scoring, verification, and retrieval explanation
/settings	Account, privacy, retention, plan, and interface preferences

Landing page content

- What Elara.ai does and which inputs it supports.
- Why scores are transparent and separated into distinct dimensions.
- Examples of completed verifications.
- The difference between Evidence Support, Attribution, and Verdict Confidence.
- Limitations of AI-assisted verification.
- Privacy, terms, retention, sharing, and correction summaries.

API separation

```
Browser
  -> Next.js frontend
    -> FastAPI authentication and validation
      -> Redis queue
        -> LangGraph worker
          -> External models, search, public APIs, and source fetcher
          -> PostgreSQL, pgvector, Redis, and object storage
```

Shareability advantage

Because Elara.ai is a web app, reports can be opened through authenticated or carefully controlled shareable links, public methodology pages can be indexed, and users receive live updates without installing software.

14. Report output, score presentation, and API contract

Example main output

Verdict: Mostly refuted
Evidence Support: 18/100
Verdict Confidence: 91/100
Source Independence: 74/100
Context Completeness: 32/100

Score	Question answered
Evidence Support	Does the collected evidence support or contradict the claim?
Attribution Support	Did the named person make the statement or communicate the alleged meaning?
Quote Fidelity	How accurately were the speaker, wording, sequence, and translation represented?
Verdict Confidence	How complete and dependable was the investigation?
Source Independence	How many genuinely separate information origins were found?
Context Completeness	Does the wording preserve definitions, dates, baselines, qualifiers, and surrounding meaning?

Completed report components

- Overview with verdict, scores, summary, limitations, and evidence timestamp.
- Atomic-claim cards with individual verdicts, confidence, importance, and gaps.
- Quote attribution, fidelity, and surrounding-context panels where applicable.
- Side-by-side supporting and contradicting evidence.
- Source-dependency graph showing originals, shared datasets, witnesses, and derivative reporting.
- Numerical audit with formulas, values, units, denominators, and reporting periods.
- Citation drawer with exact passages, page or timestamp, metadata, and source link.
- Methodology, model, prompt, workflow, parser, and source-version information.
- Feedback and correction controls.

Output contract example

```
{
  "run_id": "run_123",
  "verdict": "mostly_refuted",
  "scores": {
    "evidence_support": 18,
    "verdict_confidence": 91,
    "source_independence": 74,
    "context_completeness": 32
  },
  "atomic_claims": [...],
  "attribution": {...},
  "evidence": [...],
  "source_graph": {...},
  "calculations": [...],
  "methodology_version": "1.0"
}
```

Product language rule

Elara.ai evaluates the evidence available for a specific claim, article, or statement. It must not assign permanent honesty or credibility scores to people, companies, political groups, or publications.

15. Complete technology stack and implementation order

Complete technology stack

Category	Selected technology
Frontend framework	Next.js App Router
Frontend language	TypeScript
UI	React, Tailwind CSS, shadcn/ui
Server data	TanStack Query
Local UI state	Zustand
Forms and schemas	React Hook Form + Zod
Visualizations	React Flow + Recharts
Icons	Lucide React
Backend	FastAPI + Python + Pydantic
Persistence	SQLAlchemy + Alembic
HTTP and streaming	httpx + Server-Sent Events
Agent workflow	LangGraph
LLM	OpenAI Responses API
Search	Web-search tool or API
Embeddings	OpenAI embeddings
Tracing and evaluation	LangSmith + custom Python evaluators
Article extraction	Trafilatura + BeautifulSoup
Browser fallback	Playwright
PDF processing	PyMuPDF
Data tables	pandas
Date parsing	dateparser
Numerical auditing	Python Decimal
Main database	PostgreSQL
Vector search	pgvector
Queue and cache	Redis
Background jobs	Celery
Object storage	S3-compatible service
Frontend deployment	Vercel
API and worker deployment	Container hosting
Monitoring	Sentry
CI/CD	GitHub Actions

Recommended first milestone

Milestone

Deliver a responsive web app that accepts a text claim or article URL, creates a background verification job, streams

observable progress, and displays a citation-grounded report. Advanced provenance views, additional input types, exports, and sharing can be layered onto the same architecture.

Suggested implementation order

1. Create the Next.js application shell, design system, routes, and mocked report interface.
2. Build FastAPI authentication, verification-run schemas, and PostgreSQL persistence.
3. Add Redis, Celery, SSE progress, and a simple background verification job.
4. Implement the LangGraph claim workflow with structured outputs.
5. Add targeted retrieval, extraction, source provenance, deterministic scoring, numerical auditing, and citation auditing.
6. Finish with evaluation, monitoring, exports, sharing, corrections, and production hardening.

Architecture decision summary

Use Next.js for the web experience, FastAPI as the security and application boundary, Celery and Redis for asynchronous work, LangGraph for controlled agent orchestration, PostgreSQL with pgvector for durable and traceable evidence data, and S3-compatible object storage for larger artifacts.